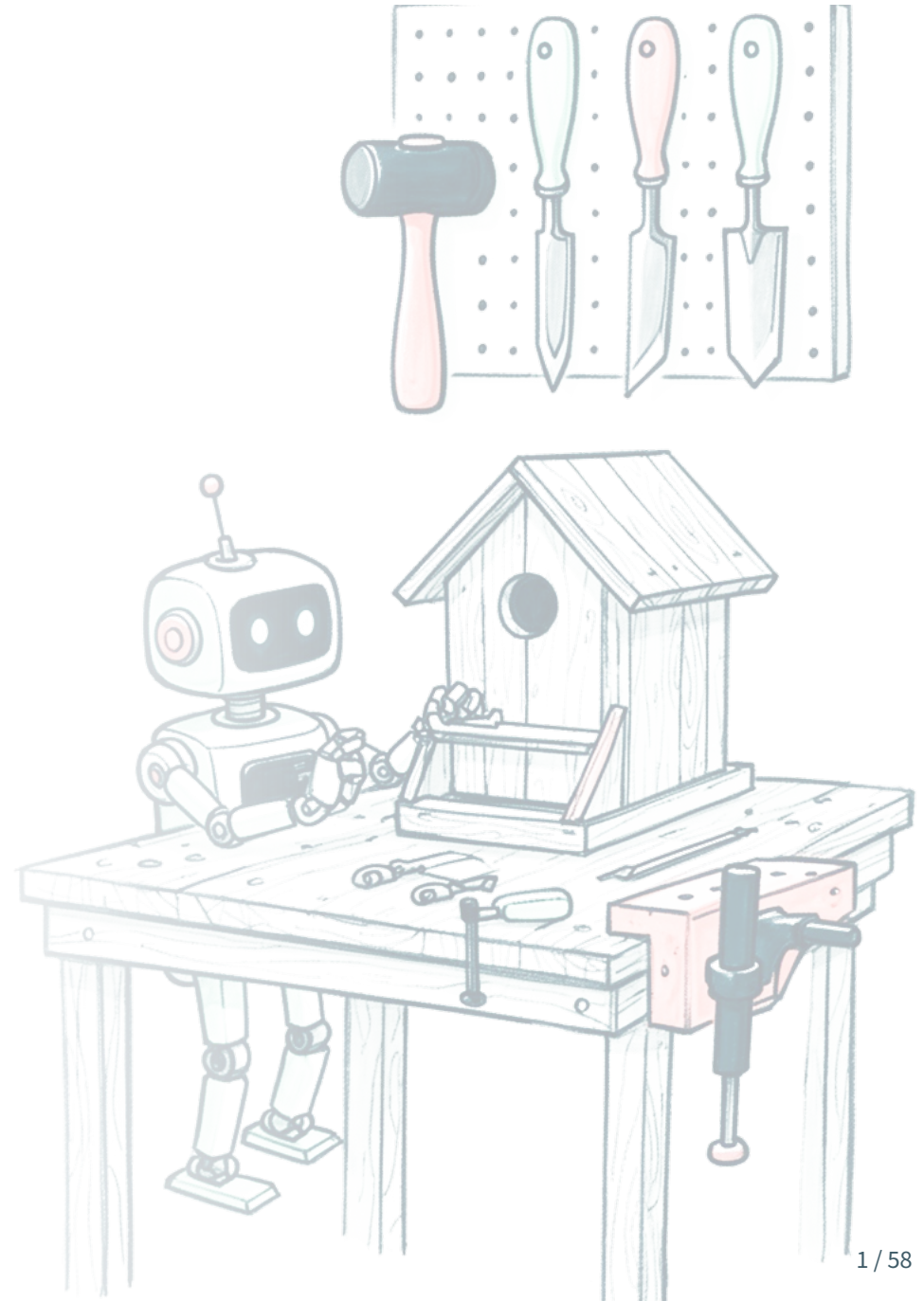


# Agentic AI in der Praxis

Vom Auftrag zur produktiven Arbeitsweise, mit Übungen am eigenen Rechner

Illustrations AI-generated · Flux 2 (flux2-dev)



# Über uns und zwei Beispiele

## Christian Uhl · ZAD, Uni Gießen

- Unbekannter Chip ohne Firmware
- Ein einzelner Fehler hätte ihn zerstört
- Sorgfältig war zu langsam, schnell zu riskant
- Nach 24 Stunden: FIDO2-Firmware, spec- und testgetrieben

## Tobias Weiß · DevOps, Uni Marburg

- openDesk Edu: Dienste für Forschung, Lehre, Verwaltung
- Wissens-Pipelines und IT-Betrieb agentisch
- Open-Source-Software mit Open-Source-Werkzeugen

# Ihre Reihe: ein paar Fragen per Handzeichen

- Wer nutzt KI-Chats wie ChatGPT regelmäßig?
- Wer hat schon einen Agenten etwas erledigen lassen?
- Wer schreibt selbst Code, und sei es nur ein Skript?
- Überlegen Sie sich ein Thema für eine Literaturrecherche

Tobias

# Der Vormittag liefert ein Werkzeug, keinen fertigen Bot

- Ein Research-Repo mit laufender Pipeline
- Einen prüfbaren Auftrag, live umgesetzt
- Werkzeuge für Modelle und Harnesses
- Nicht dabei: der voll-autonome Assistent

Tobias

# Bis zur ersten Pause: Grundlagen, Modelle, Werkzeuge

## Block

Ankommen, Vorstellung, Handzeichen-Runde

Grundlagen: Agenten beauftragen

Foundation Models: aktuelle Entwicklungen

Open-Source Toolbox

☹ Pause 10:10 bis 10:20

Tobias

# Danach zweimal selbst ausprobieren, dazwischen der Weg zur Spezifikation

## Block

Hands-On: Erste Schritte, direkt agentisch

Vom Auftrag zur Spezifikation, und was sie an Token spart

Hands-On: Mit Spec

☕ Pause 11:25 bis 11:30

Outcomes: Ihre Ergebnisse

Ihre Anwendungsfälle, Q&A, Diskussion

Christian

1

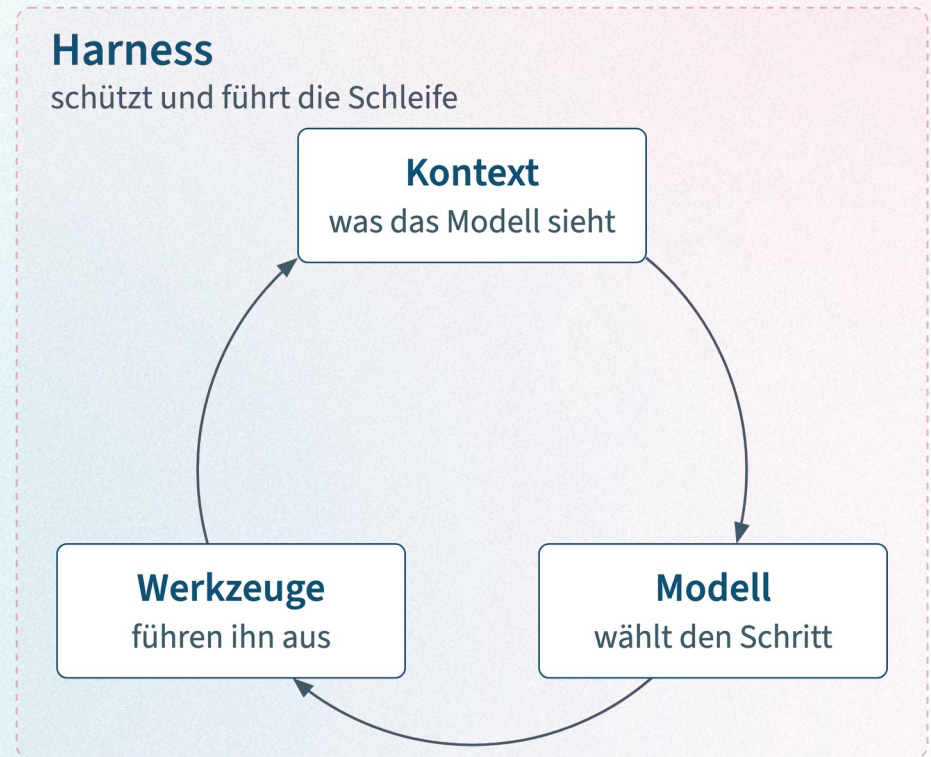
# Grundlagen: worüber wir sprechen

# Bei diesen fünf Begriffen legen wir uns für heute fest

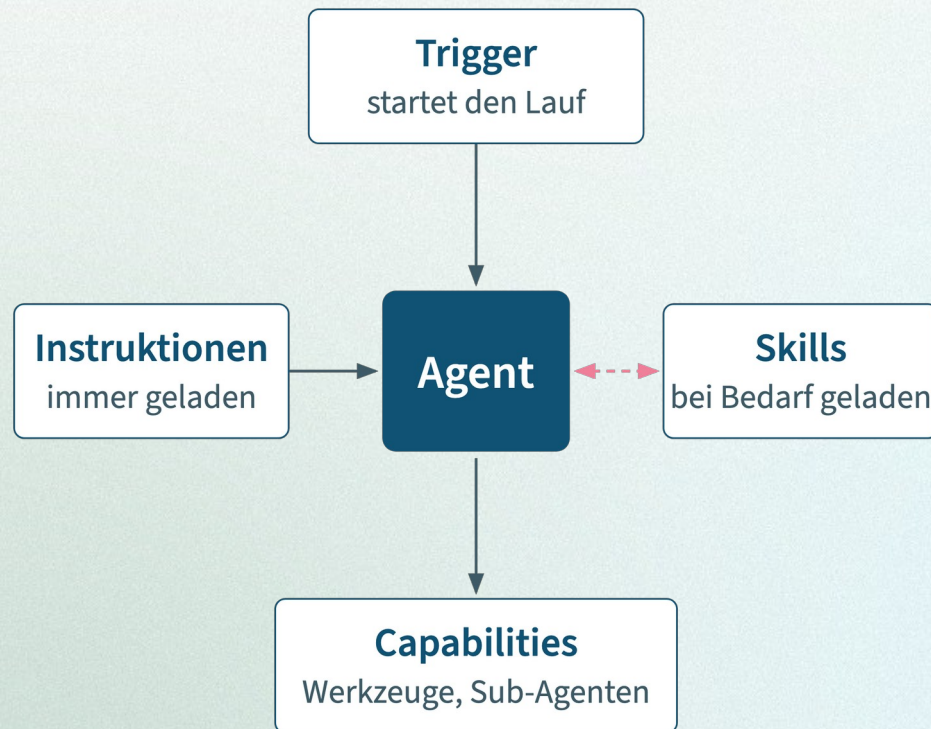
| Begriff            | Worüber gestritten wird                | Unsere Festlegung                         |
|--------------------|----------------------------------------|-------------------------------------------|
| Agent              | von Chatbot bis autonomes System       | Das Modell bestimmt die nächsten Schritte |
| Reasoning          | ob das Denken ist                      | mehr Rechenzeit zur Laufzeit              |
| Halluzination      | Begriff umstritten, eher Konfabulation | unbelegte Ausgabe, Grund für die Tests    |
| Open Weights       | Gewichte offen, Rest oft nicht         | Lizenzen getrennt prüfen                  |
| Reproduzierbarkeit | gleiche Eingabe, anderes Ergebnis      | Stand sichern, Analyse wiederholen        |

# Ein agentisches System besteht aus vier Bausteinen

- Modellwissen ist keine Datenbank
- Kontext ist Arbeitsgedächtnis
- Werkzeuge nur, soweit freigegeben
- Das Harness setzt Grenzen



# Wer einen Agenten aufbaut, legt vier Dinge fest



- Instruktionen: AGENTS.md, verweisen auf Skills
- Skills: Anleitungen zum Nachschlagen
- Trigger: Befehl, Zeitplan oder Ereignis
- Capabilities legen fest, was er darf

Christian

The screenshot shows the 'Build' tab of an AI agent configuration interface. It features five main sections: 'Model', 'Instructions', 'Skills', 'Triggers', and 'Capabilities'. The 'Model' section has a dropdown menu set to 'qwen38-flash-next' from an 'OpenAI account'. The 'Instructions' section contains a text box with the instruction 'Summarize new tickets in the Customer Support Slack channel.' The 'Skills' section has a text box and an 'Add skill' button. The 'Triggers' section has a text box, 'Add channel' and 'Add schedule' buttons, and a 'Preview' button. The 'Capabilities' section has a text box and 'Add tool', 'Add workflow', and 'Add sub-agent' buttons. At the bottom, there is a 'Web search' toggle set to 'Off'.

**Build** Knowledge Sessions Settings

**Model \***  
What model this agent uses  
qwen38-flash-next OpenAI account

**Instructions \***  
What this agent should do  
Summarize new tickets in the Customer Support Slack channel.

**Skills**  
How this agent handles specific tasks  
Add skill

**Triggers**  
Where and when this agent runs  
Add channel  
Add schedule  
Preview

**Capabilities**  
What this agent can do and how it does it  
Add tool  
Add workflow  
Add sub-agent

**Web search**  
Let the model search the web when it needs up-to-date information.  
Off

**Auch Agents as a Service brauchen genau diese vier Dinge**

Christian

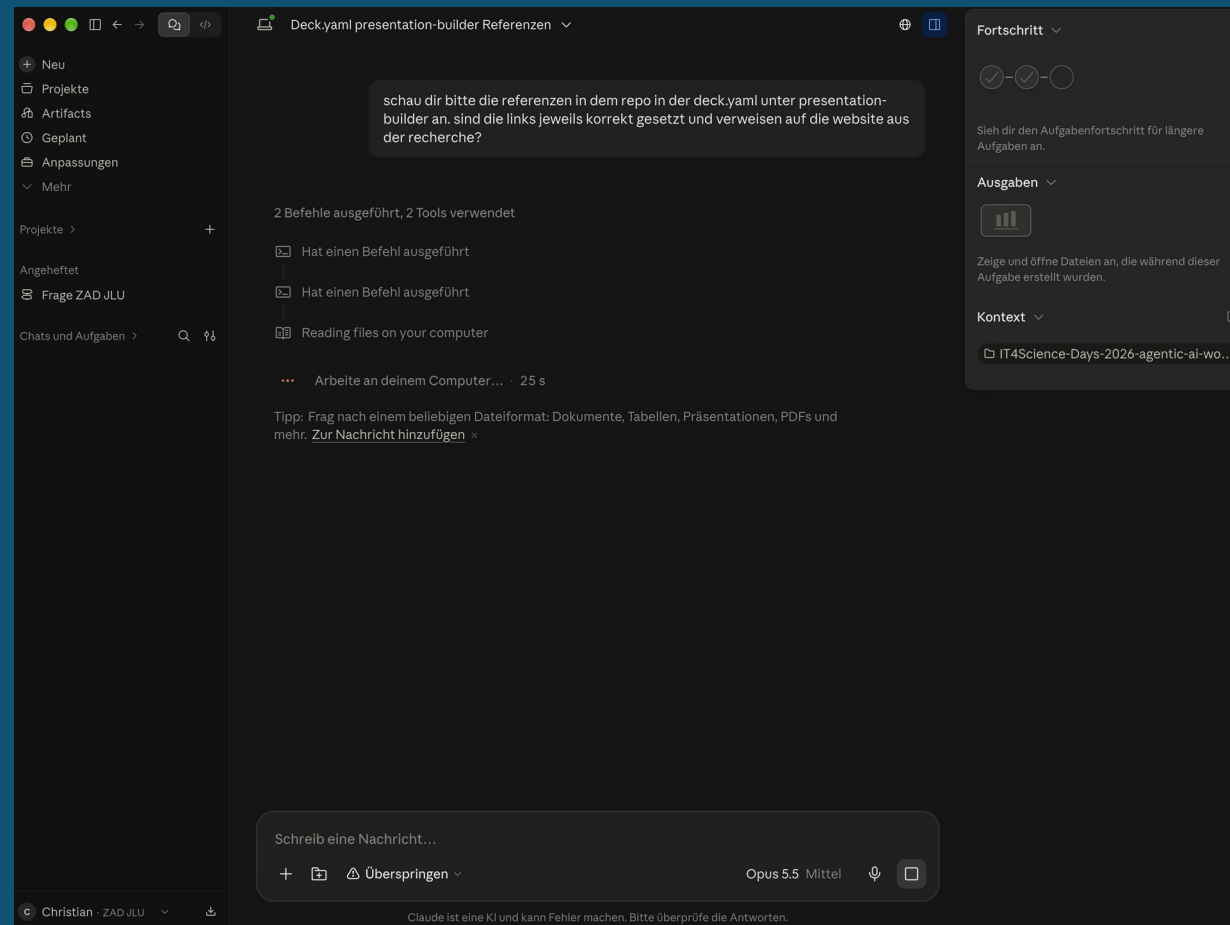
# Ein Skill ist eine Anleitung, die das Modell selbst nachschlägt

- Ein Ordner mit SKILL.md: Name, Beschreibung, Anleitung
- Im Kontext steht anfangs nur die Beschreibung
- Das Modell entscheidet, wann es nachliest
- Offener Standard, läuft in OpenCode und pi
- Fremde Skills vorher lesen, sie können Code ausführen

# Wo wir Agenten heute schon begegnen

| Wo        | Beispiel                         | Der Agent entscheidet selbst             |
|-----------|----------------------------------|------------------------------------------|
| Recherche | Deep Research in Gemini, ChatGPT | welche Quelle er als Nächstes liest      |
| Büro      | Copilot Researcher in M365       | was er in Mails und Dateien sucht        |
| Browser   | Claude in Chrome, Comet          | wohin er klickt, was er einträgt         |
| Zuhause   | Alexa+                           | welchen Dienst er für eine Buchung nutzt |
| Forschung | Elicit, Google Co-Scientist      | wie er eine Forschungsfrage zerlegt      |
| Software  | Claude Code, Codex, Copilot      | welche Datei er ändert, was er testet    |

Christian



## Beispiel: Claude Cowork

Christian

2

# Wie man einen Agenten beauftragt

# Fragen oder Beauftragen



- Fragen: „Fassen Sie diese Papers zusammen.“
- Beauftragen: „Extrahieren Sie diese fünf Felder aus jedem Paper.“

# Ein prüfbarer Auftrag hat drei Ebenen

- Jede Ebene ist eine Datei, kein Gespräch
- Der Vertrag ist eine Spezifikation für eine Änderung
- Der Vertrag nennt das Kriterium, der Test wendet es an



# Jede der drei Ebenen legt etwas anderes fest

| Ebene         | Legt fest                                     | Im Repository                         |
|---------------|-----------------------------------------------|---------------------------------------|
| Spezifikation | was gelten soll, prüfbar formuliert           | specs/, AGENTS.md                     |
| Vertrag       | was sich jetzt ändert, und woran man es misst | ein Change mit Vorschlag und Aufgaben |
| Tests         | ob die geprüften Fälle bestehen               | Testsuite, Validierung, CI            |

Christian



**Erledigt ist eine Aufgabe erst,  
wenn die Prüfung bestanden ist.**

Christian

3

# Foundation Models: aktuelle Entwicklungen

# Sechs Entwicklungen prägen die Modelle im Jahr 2026

- Reasoning ist reif: planen statt vervollständigen
- Kontext wächst von 200.000 auf über eine Million Token
- MCP wird der Standard für Werkzeugzugriff
- Werkzeugaufrufe sind produktionsreif
- Offene Gewichte holen auf, die Kosten fallen
- Lokal und souverän betreiben ist eine echte Option

Christian

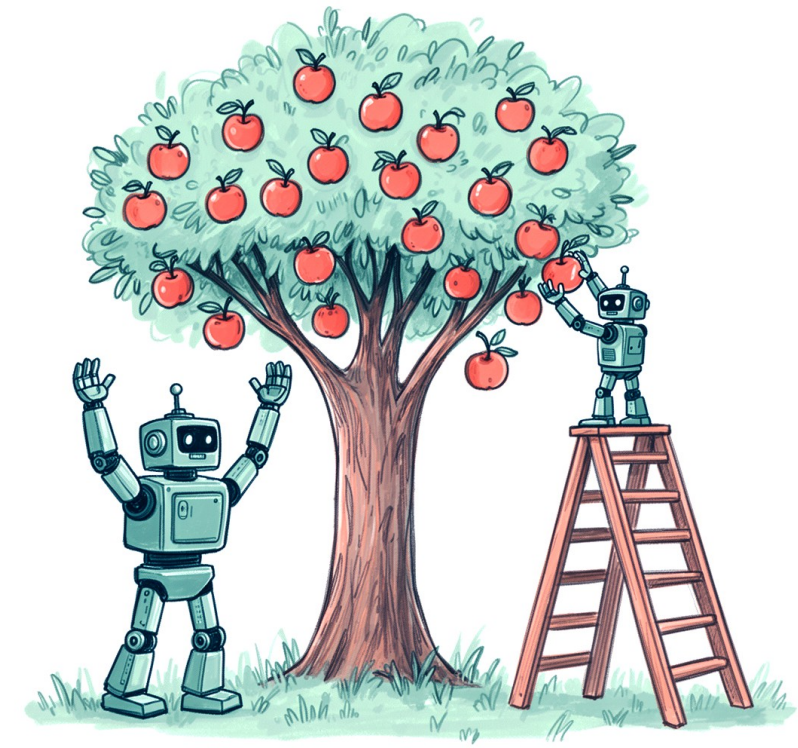
# Unsere Modellwahl im Alltag

| Aufgabe                  | Unsere Wahl                      | Einordnung                             |
|--------------------------|----------------------------------|----------------------------------------|
| Freie Texte, Übersetzung | GPT-6 Astra                      | bewährt in unseren Schreibaufgaben     |
| Vision                   | GPT-6 Astra, Claude Fable 5      | Fable 5 führt die Arena Vision         |
| Code, größere Umbauten   | Claude Opus 5.5, zunehmend GLM   | bei uns oft wenige Korrekturen         |
| Code gezielt umschreiben | Qwen3.8-Flash-Next, GLM          | offene Gewichte, schnell, günstig      |
| Abläufe mit Werkzeugen   | GLM 5.3 Flash, Claude Sonnet 5.5 | verlässliche Werkzeugaufrufe           |
| Sensible Forschungsdaten | Qwen3.8 27B, lokal via vLLM      | die Daten verlassen das Haus nicht     |
| Viele gleichartige Fälle | DeepSeek V4.1 Flash              | Geschwindigkeit und Kosten entscheiden |

Christian

# Das Harness bestimmt, was das Modell erreichen kann

- Modell und Kontext prägen die Antwort
- Das Harness begrenzt die Ausführung
- Das Modell schlägt nächste Schritte vor
- Gutes Harness kann Modellschwächen ausgleichen

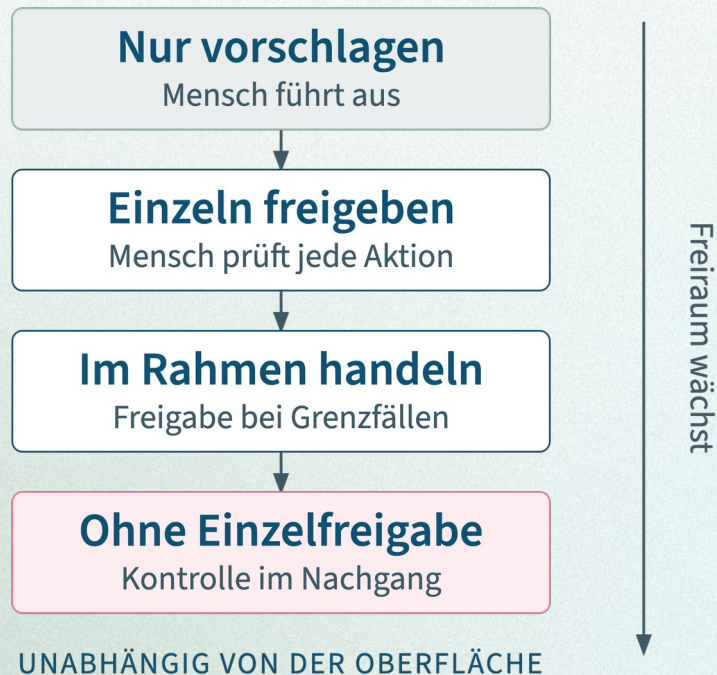


Tobias

4

# Open-Source Toolbox: Harnesses als Steuerungsebene

# Agentisch ist nicht das Terminal, sondern die Autonomie



- Auch eine GUI kann agentisch sein
- Autonomie heißt: delegierte Befugnisse
- CI automatisiert, sie entscheidet nicht selbst

Tobias

# Der Agent arbeitet, die Verantwortung bleibt bei Ihnen

- Sie verstehen, was der Agent getan hat
- Sie geben frei, der Agent schlägt nur vor
- Geht etwas schief, stehen Sie dafür ein
- Ungeprüft nur, wo ein Fehler nichts kostet

# Zwei offene Harnesses, der Einsatz entscheidet

## OpenCode, der Allrounder

- Agent im Terminal, Modell je Agent
- LSP, Plugins, Skills, MCP
- oh-my-opencode: Routing, AST-Grep, Background-Agents
- Für tägliche Coding-Agents

## pi, der Minimalist

- Skills, Prompt-Templates, Packages
- „Adapt pi, nicht umgekehrt“
- Für schlanke Setups mit voller Kontrolle

# pi bleibt klein, alles andere kommt als Package

| Kategorie              | Beispiele                                     |
|------------------------|-----------------------------------------------|
| Skills                 | superpowers, CVE-Scanner, AGENTS.md-Audit     |
| Agenten                | subagents, until-done, plan-mode, evaluate    |
| Stil, Werkzeuge, Suche | caveman, ponytail, mcp-Adapter, websearch     |
| UI                     | webui, desktop-ui, glimpseui, chrome          |
| Absicherung            | tool-repair, retry, permissions, secrets, rtk |

# Für den Start empfehlen wir diese acht Packages

| Package                  | Leistet                                      |
|--------------------------|----------------------------------------------|
| pi-plan-mode             | liest und plant nur, bis Sie freigeben       |
| pi-permission-system     | legt fest, was der Agent ohne Rückfrage darf |
| pi-secrets               | Zugangsdaten erreichen das Modell nie        |
| superpowers              | Skills für Planen, Testen und Fehlersuche    |
| pi-subagents             | verteilt Teilaufgaben an eigene Agenten      |
| pi-websearch             | Websuche als Werkzeug des Agenten            |
| pi-tool-repair, pi-retry | fangen fehlerhafte Aufrufe und Abbrüche ab   |
| pi-rtk                   | kürzt Shell-Ausgaben und spart Token         |

---

Pause.  
**Zehn Minuten.**



Tobias

5

# Hands-On: Erste Schritte

# Erst planen, dann bauen

- Plan-Mode: der Agent liest, fragt und schlägt vor
- Nichts wird geändert, bis Sie zustimmen
- Build-Mode: der Agent setzt den freigegebenen Plan um
- Den Plan lesen ist der schnellste Weg zum Verständnis

Tobias

# Übung 1: Ihr Thema X, direkt an den Agenten, fünfzehn Minuten

## Der Agent übernimmt

## Heißt konkret

Entdecken

neue Veröffentlichungen zu Thema X aufspüren

Einordnen

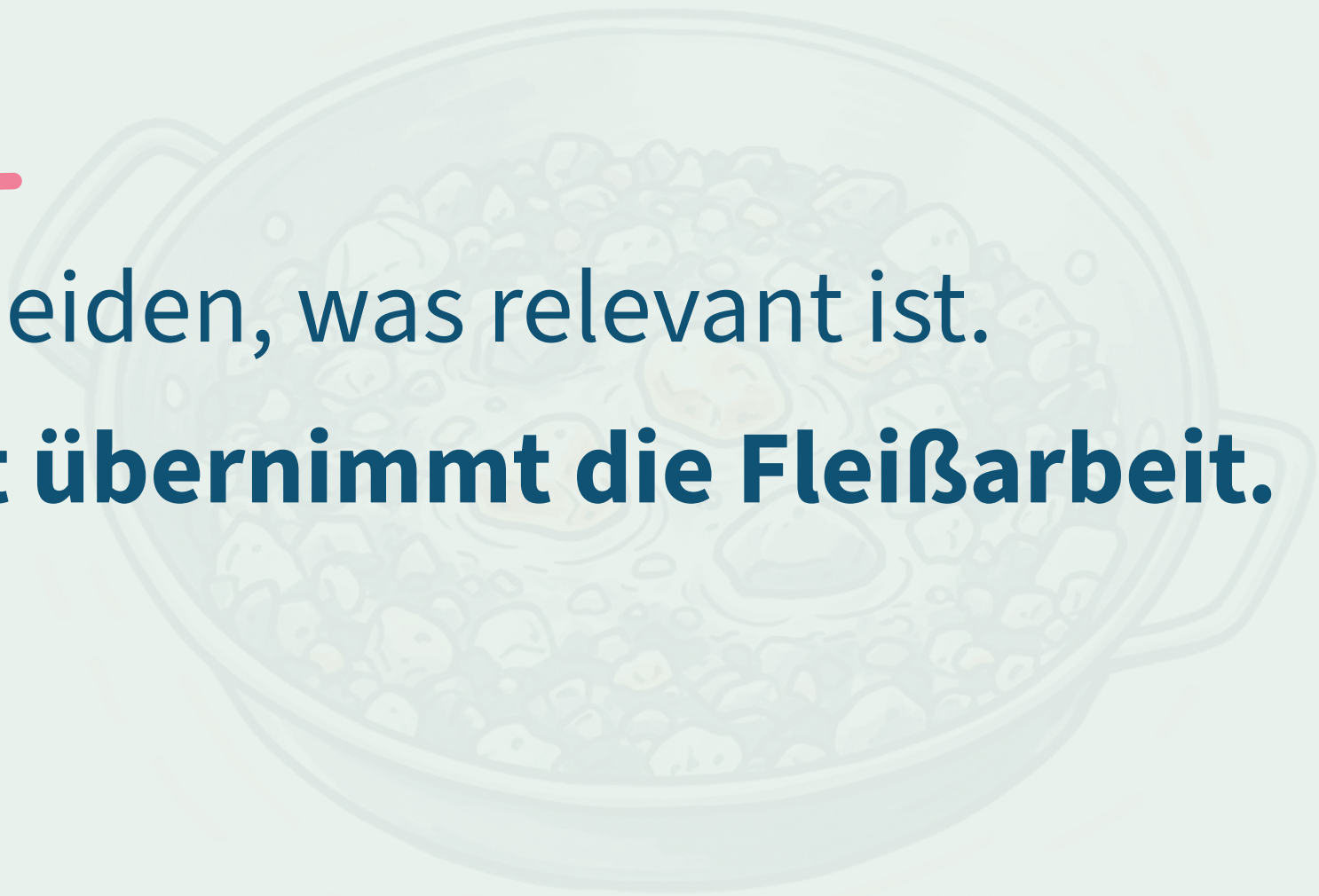
Funde den Themenfeldern zuordnen

Prüfen

Quellen verifizieren, Erfindungen aussortieren

Berichten

Überblick, Trends, Kurz-Briefings



Sie entscheiden, was relevant ist.  
**Der Agent übernimmt die Fleißarbeit.**

Tobias

# AGENTS.md: Regeln, die der Agent bei jedem Start liest

- Erzeugtes wird nie von Hand geändert
- Nichts wird erfunden, jede Quelle muss auflösbar sein
- Nach jeder Änderung die komplette Prüfung, dann Commit
- Geführt per Datei, nicht per Prompt

6

# Vom Auftrag zur Spezifikation, und was sie an Token spart

# **In diesem Kapitel wird aus dem Auftrag eine Datei**

- Eine Vorlage: ein fertiger Change zum Nachbauen
- Wo Spezifikation, Vertrag und Archiv liegen
- Spec-Kit oder OpenSpec: warum wir OpenSpec nehmen
- Warum eine präzise Spezifikation Token spart

# Ein fertiger Change steht als Vorlage bereit

- Repo ai-literacy-research: über 9.600 Papers, 20 Kategorien
- openspec/specs/: Corpus, Discovery, Analyse
- Change add-research-gap-analysis: proposal, design, specs, tasks
- Gap-Score:  $0,6 \times \text{Dünne} + 0,4 \times \text{Momentum} + \text{Floor-Bonus}$

# Wo Spezifikation, Auftrag und Tests liegen

| Ort               | Was dort liegt                  | Rolle                                |
|-------------------|---------------------------------|--------------------------------------|
| openspec/specs/   | das dauerhaft gültige Verhalten | Spezifikation                        |
| openspec/changes/ | die eine anstehende Änderung    | Vertrag, genau das bekommt der Agent |
| tests/            | Prüfungen der Anforderungen     | Tests, ausgeführt lokal oder in CI   |

# Spec-Kit baut Neues, OpenSpec ändert Bestehendes

|                     | Spec-Kit                       | OpenSpec                                 |
|---------------------|--------------------------------|------------------------------------------|
| Grundidee           | was gebaut werden soll         | wie sich Bestehendes ändert              |
| Typischer Fall      | Neubau, neue Funktionen        | laufende Weiterentwicklung               |
| Quelle der Wahrheit | eine Spezifikation je Vorhaben | dauerhafte Spezifikation plus Änderungen |
| Ablauf              | specify, clarify, plan, tasks  | propose, apply, sync, archive            |
| Aufwand             | höher, mehr Prozessführung     | niedriger                                |

# Token, Kosten und Qualität getrennt messen

- Routing: Kosten pro gelöster Aufgabe
- Caching: Berechnung wiederverwenden
- Kontext: nur Relevantes mitgeben
- Specs können Korrekturschleifen sparen
- Qualität an festen Kriterien prüfen

Christian

7

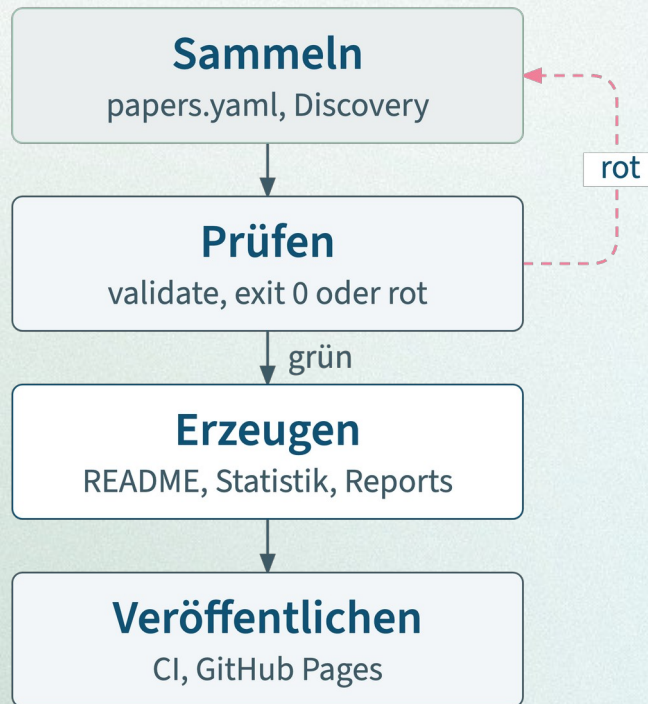
# Hands-On: Mit Spec

Tobias

# Jeder geht mit einem eigenen, CI-validierten Research-Repo

- Gleiche Frage wie in Übung 1, jetzt mit Spec
- Corpus: strukturierter Literaturbestand
- Regeln statt Handarbeit: prüfen, erzeugen, berichten
- Lebendig statt Momentaufnahme: CI prüft wöchentlich

# In der Pipeline hat jede Stufe genau eine Aufgabe



- Ein Werkzeug, ein Job
- Text ist die Schnittstelle, für Mensch und Agent
- Grün heißt: definierte Checks bestanden

# Der Jump-Start dauert fünf Schritte

- Kopie holen: das Demo-Repo wird Ihres
- Themenfeld setzen: Ihre Ordnung für Thema X
- Seeden: drei bis fünf Treffer aus Übung 1
- Laufen lassen: prüfen, erzeugen, berichten
- Veröffentlichen: das Ergebnis geht online

Christian

## Übung 2: eine überprüfbare Anforderung für Ihr Thema

- Klein halten: eine Auswertung, eine Kennzahl
- Fertig: Tests bestanden, Ergebnis geprüft
- Mit Agent: erst /opsx-propose, dann /opsx-apply
- Ohne OpenSpec: dieselbe Struktur als Markdown

# Übung 2 Schritt für Schritt, am Beispiel thema-x

| Schritt      | Eingabe                                                          | Ergebnis                           |
|--------------|------------------------------------------------------------------|------------------------------------|
| 1 · Terminal | <code>openspec init --tools opencode,pi</code>                   | Befehle und Skills für den Agenten |
| 2 · Agent    | <code>/opsx-propose thema-x: Papers pro Jahr als Kennzahl</code> | Proposal, Tasks und Spec           |
| 3 · Terminal | <code>openspec status --change thema-x</code>                    | welche Teile schon stehen          |
| 4 · Terminal | <code>openspec validate thema-x --strict</code>                  | Strukturprüfung der Spec           |
| 5 · Agent    | <code>/opsx-apply thema-x</code>                                 | der Agent arbeitet die Tasks ab    |
| 6 · Terminal | <code>openspec list</code>                                       | Fortschritt, etwa 4/4 tasks        |
| 7 · Agent    | <code>/opsx-archive thema-x</code>                               | die Spec wird fortgeschrieben      |

# Schon das Mindestziel zeigt, dass die Methode sitzt

| Ebene         | Mindestziel                           | Vollständig                                 |
|---------------|---------------------------------------|---------------------------------------------|
| Spezifikation | ein Satz, der widerlegt werden kann   | Szenario: Ausgangslage, Eingriff, Ergebnis  |
| Vertrag       | eine Aufgabenliste mit drei Punkten   | Vorschlag, Entwurf, Spezifikation, Aufgaben |
| Tests         | eine Prüfung, die fehlschlagen könnte | läuft automatisch und besteht               |

---

Pause.

**Fünf Minuten.**



Tobias

8

# Outcomes: Ihre Ergebnisse

## Gezeigt wird ein eigenes Repo mit eigenem Change

- Drei bis vier Freiwillige, je drei bis vier Minuten
- Pipeline grün, Change validiert, live oder als Screenshot
- Bewertet an der Pyramide: Spec, Change, grüner Check
- Das Publikum reviewt mit

Tobias

9

# Ihre Anwendungsfälle, Q&A, Diskussion

# So übertragen Sie das Muster auf eigene Projekte

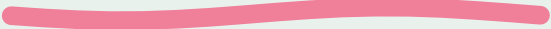
- Klein starten: eine wiederkehrende Aufgabe, ein Prüfkriterium
- Den Auftrag als Datei: Spec, erster Change mit drei Tasks
- Checks bestehen, Ergebnis fachlich prüfen
- Instruktionen und Skills anlegen statt jedes Mal erklären
- Erkenntnisse aus dem Betrieb werden neue Changes

# Die Werkzeuge des Vormittags zum Nachinstallieren

| Werkzeug                 | Adresse                                                                                                                     |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| OpenCode                 | <a href="https://github.com/sst/opencode">github.com/sst/opencode</a>                                                       |
| pi                       | <a href="https://pi.dev">pi.dev</a>                                                                                         |
| zot                      | <a href="https://zot.sh">zot.sh</a>                                                                                         |
| OpenSpec                 | <a href="https://npmjs.com/package/@fission-ai/openspec">npmjs.com/package/@fission-ai/openspec</a>                         |
| SAIA-Plugin für pi       | <a href="https://codeberg.org/tobias-weiss-ai-xr/pi-saia-plugin">codeberg.org/tobias-weiss-ai-xr/pi-saia-plugin</a>         |
| SAIA-Plugin für OpenCode | <a href="https://github.com/tobias-weiss-ai-xr/opencode-saia-plugin">github.com/tobias-weiss-ai-xr/opencode-saia-plugin</a> |
| oh-my-opencode           | <a href="https://github.com/code-yeongyu/oh-my-opencode">github.com/code-yeongyu/oh-my-opencode</a>                         |

# Repositories und Skills zum Weiterarbeiten

| Name                 | Adresse                                                                                                                     |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| skeleton-research    | <a href="https://github.com/tobias-weiss-ai-xr/skeleton-research">github.com/tobias-weiss-ai-xr/skeleton-research</a>       |
| ai-literacy-research | <a href="https://github.com/tobias-weiss-ai-xr/ai-literacy-research">github.com/tobias-weiss-ai-xr/ai-literacy-research</a> |
| Superpowers          | <a href="https://github.com/obra/superpowers">github.com/obra/superpowers</a>                                               |
| rtk                  | <a href="https://github.com/rtk-ai/rtk">github.com/rtk-ai/rtk</a>                                                           |
| ponytail             | <a href="https://github.com/DietrichGebert/ponytail">github.com/DietrichGebert/ponytail</a>                                 |
| caveman              | <a href="https://github.com/JuliusBrussee/caveman">github.com/JuliusBrussee/caveman</a>                                     |



# Vielen Dank!

# Fragen gern als GitHub-Issue.

CC BY 4.0 · Christian Uhl · Tobias Weiß

# Quellen und Links

- [1] Model Context Protocol — <https://modelcontextprotocol.io>
- [2] Agent Skills Specification — <https://agentskills.io/specification>
- [3] OpenCode Skills — <https://opencode.ai/docs/skills/>
- [4] pi Skills — <https://pi.dev/docs/latest/skills>
- [5] Snyk ToxicSkills, 02/2026 — <https://snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub/>
- [6] Gemini Deep Research — <https://gemini.google/de/overview/deep-research/>
- [7] Microsoft 365 Copilot Researcher — <https://learn.microsoft.com/en-us/copilot/microsoft-365/faq-researcher>
- [8] Claude in Chrome — <https://claude.com/blog/claude-in-chrome-generally-available>
- [9] Alexa+ in Deutschland — <https://www.teltarif.de/en/alexaplus-deutschland-amazon/news/103392.html>
- [10] Elicit Research Agent — <https://elicit.com/blog/introducing-research-agent-workflows>
- [11] Google Co-Scientist — <https://deepmind.google/blog/co-scientist-a-multi-agent-ai-partner-to-accelerate-research/>
- [12] Arena Vision Leaderboard, Stand 28.09.2026 — <https://arena.ai/leaderboard/vision>
- [13] Ewerlöf, Coding is NOT Solved, 26.09.2026 — <https://blog.alexewerlof.com/p/coding-is-not-solved>
- [14] 26 Packages, eigene Installation, Stand 09/2026 — <https://pi.dev/packages>
- [15] Alle Packages: pi.dev/packages — <https://pi.dev/packages>
- [16] Option für SAIA: pi-saia-plugin — <https://github.com/tobias-weiss-ai-xr/pi-saia-plugin>
- [17] GitHub Spec Kit — <https://github.com/github/spec-kit>
- [18] Fission AI OpenSpec — <https://github.com/Fission-AI/OpenSpec>
- [19] Prompt Caching — <https://developers.openai.com/api/docs/guides/prompt-caching>
- [20] OpenSpec 1.13.2, CLI-Referenz — <https://github.com/Fission-AI/OpenSpec/blob/main/docs/cli.md>

# Quellen und Links (2)

- [21] OpenCode – <https://github.com/sst/opencode>
- [22] pi – <https://pi.dev>
- [23] zot – <https://zot.sh>
- [24] OpenSpec – <https://www.npmjs.com/package/@fission-ai/openspec>
- [25] pi-saia-plugin – <https://codeberg.org/tobias-weiss-ai-xr/pi-saia-plugin>
- [26] opencode-saia-plugin – <https://github.com/tobias-weiss-ai-xr/opencode-saia-plugin>
- [27] oh-my-opencode – <https://github.com/code-yeongyu/oh-my-opencode>
- [28] skeleton-research – <https://github.com/tobias-weiss-ai-xr/skeleton-research>
- [29] ai-literacy-research – <https://github.com/tobias-weiss-ai-xr/ai-literacy-research>
- [30] Superpowers – <https://github.com/obra/superpowers>
- [31] rtk – <https://github.com/rtk-ai/rtk>
- [32] ponytail – <https://github.com/DietrichGebert/ponytail>
- [33] caveman – <https://github.com/JuliusBrussee/caveman>